GOSIM

GOSIM Shenzhen 2026

vLLM EXTERNAL PLUGIN

AFD Plugin

Attention - FFN Disaggregation in vLLM



CONTENTS

Part 01 背景与项目概览

Part 02 架构与运行机制

Part 03 部署方式与性能结果

Part 04 社区进展与后续规划



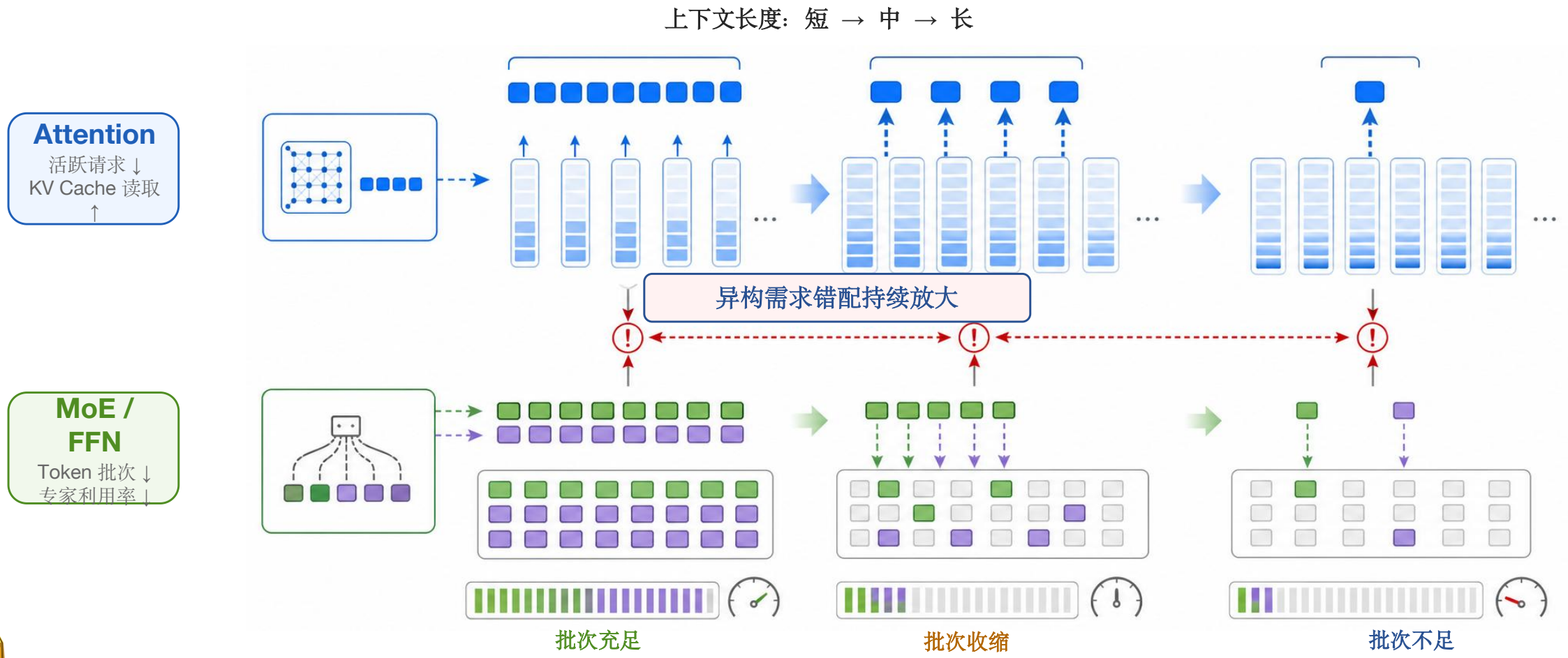
01

背景与项目概览



Attention 与 MoE 的资源需求越不匹配

Attention 受 KV Cache 容量约束，而 MoE 依赖大批次提升专家 GEMM 效率。

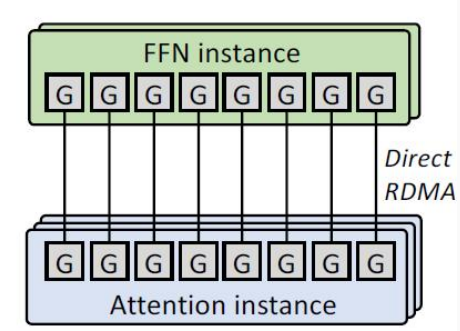


上下文增长使 Attention 更受内存约束，同时削弱 MoE 的批处理效率

来源：用户提供的 AFD Plugin 动机说明

Attention – FFN 解耦带来弹性配比、异构部署与低时延

方法：拆分计算角色

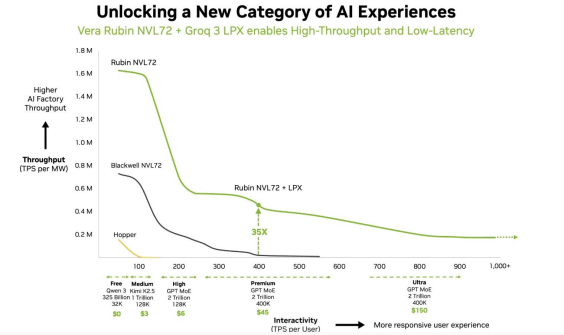


N × FFN

M × Attention

将 Attention 与 FFN 部署到独立集群，
通过 RDMA / NVLink / UB 交换激活。

外部趋势：吞吐与交互性同时优化

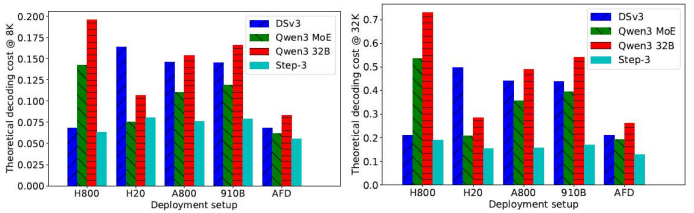


01 弹性配比

解耦后可独立调整 A:F 实例比例，突破共置场景的共同 batch 上限。



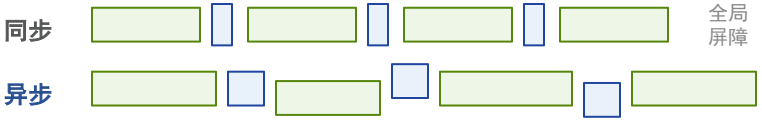
02 异构部署



Attention 与 FFN 可分别选择成本效率更优的硬件。

异构组合具备
进一步压低 decode cost 的空间。

03 低时延



异步调度消除全局同步屏障，
让通信与计算重叠，减少 idle waiting。

来源：原始 PPT 第 1 页（Methodology / Benefits）

AFD Plugin 是 vLLM 的 Attention－FFN 解耦外部插件

github.com/vllm-project/afd-plugin

将 Attention 与 FFN / MoE 拆为独立运行角色，通过 Connector 交换激活，并让两侧资源按负载独立扩缩。

插件化接入

通过 vllm.general_plugins、role worker、model wrapper 与 additional_config 接入。

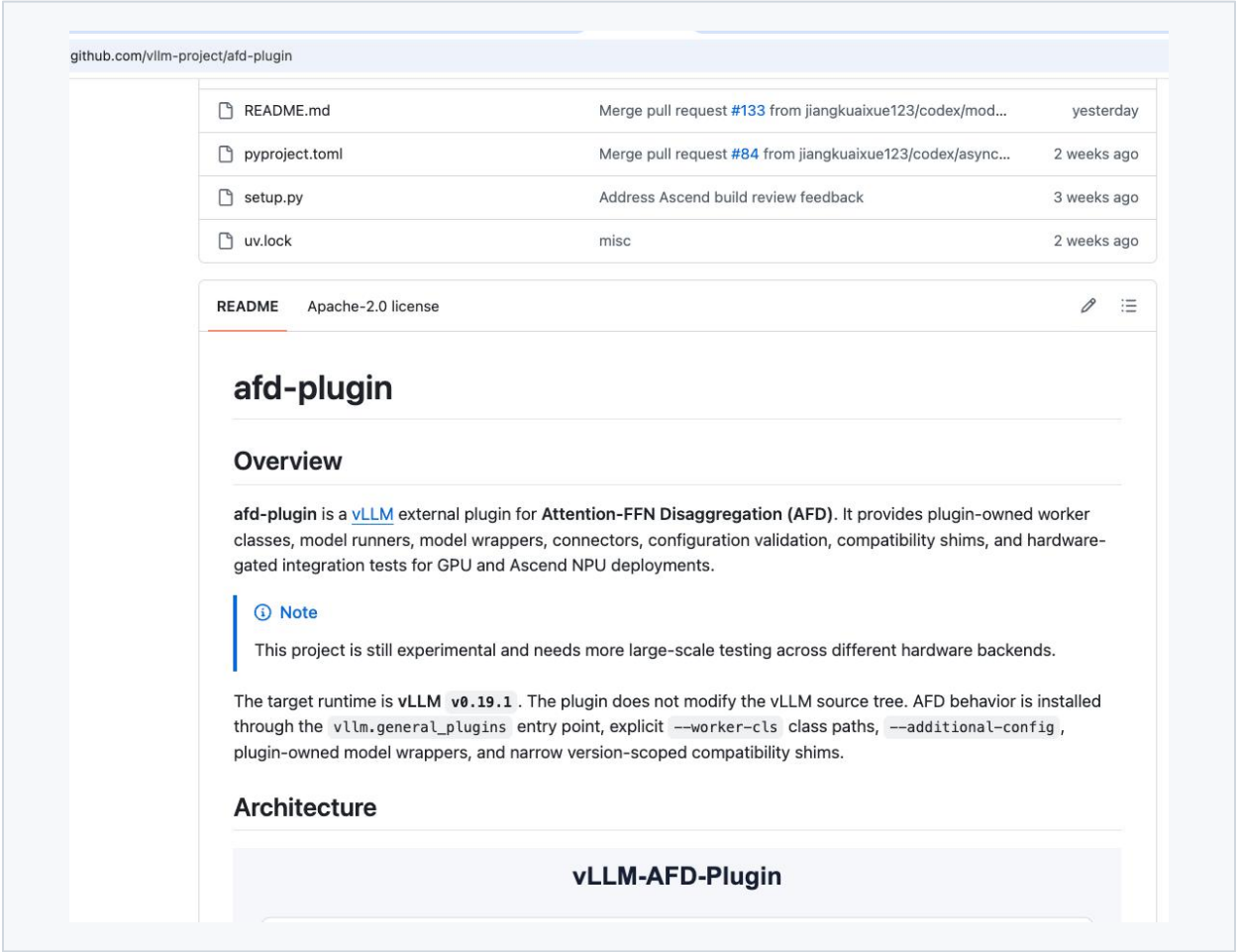
多硬件、多模式

覆盖 CUDA 与 Ascend NPU，支持同步 Decode 与异步 Prefill Connector。

明确的兼容边界

不修改 vLLM/vLLM-Ascend source tree。

当前状态：experimental，仍需在更多硬件与大规模场景中验证。



来源：README.md；https://github.com/vllm-project/afd-plugin

02

架构与运行机制



模块组成：两类角色 + 一层 Connector，以插件接入 vLLM

接入点
vLLM 扩展点

- **插件入口**：general_plugins → register_afd
- **配置**：--additional-config {"afd": ⋯}
- **Worker 选择**：按 role × 平台自动选择
- **模型注册**：ModelRegistry 注册 AFD 模型

Attention 角色
请求驱动 · 调度 / KV Cache / 采样

Worker
CUDA · AFDAttentionWorker
Ascend · AFDNPUAttentionWorker

ModelRunner
CUDA · AFDAttentionModelRunnerV2
Ascend · AFDNPUAttentionModelRunner / V2

AFD 元数据
Stage / DP / Graph 状态写入 ForwardContext
供模型层与 Connector 读取

AFD Connector
A2F / F2A 双向传输 · 可选控制面

契约
AFDConnectorBase：生命周期 + 双向数据面
AFDControlPlane：同步路径的控制面

CUDA
P2pNcclAFDConnector
NCCL P2P · 同步

Ascend NPU
CAMP2pAFDConnector · CAM P2P · 同步
CAMAsyncAFDConnector · CAM 异步

FFN 角色
Connector 驱动 · 无 Scheduler / KV Cache

EngineCore（Patch）
跳过 Scheduler 与 KV Cache 初始化
进入后台 daemon 循环

Worker
CUDA · AFDFFNWorker
Ascend · AFDNPUFFNWorker

ModelRunner
CUDA · GPUFFNModelRunner
Ascend · AFDNPUFFNModelRunner

模型封装
按角色只构建、加载本侧模块

DeepSeek V2 / V3 / V3.2 | DeepSeek V4 | Qwen3 MoE（CUDA） | Qwen3.5 / 3.6 MoE（CUDA）

兼容层与算子
收敛上游差异

compat/patches vLLM 兼容 Patch
compat/npu vLLM-Ascend 适配
csrc/npu CANN 自定义算子

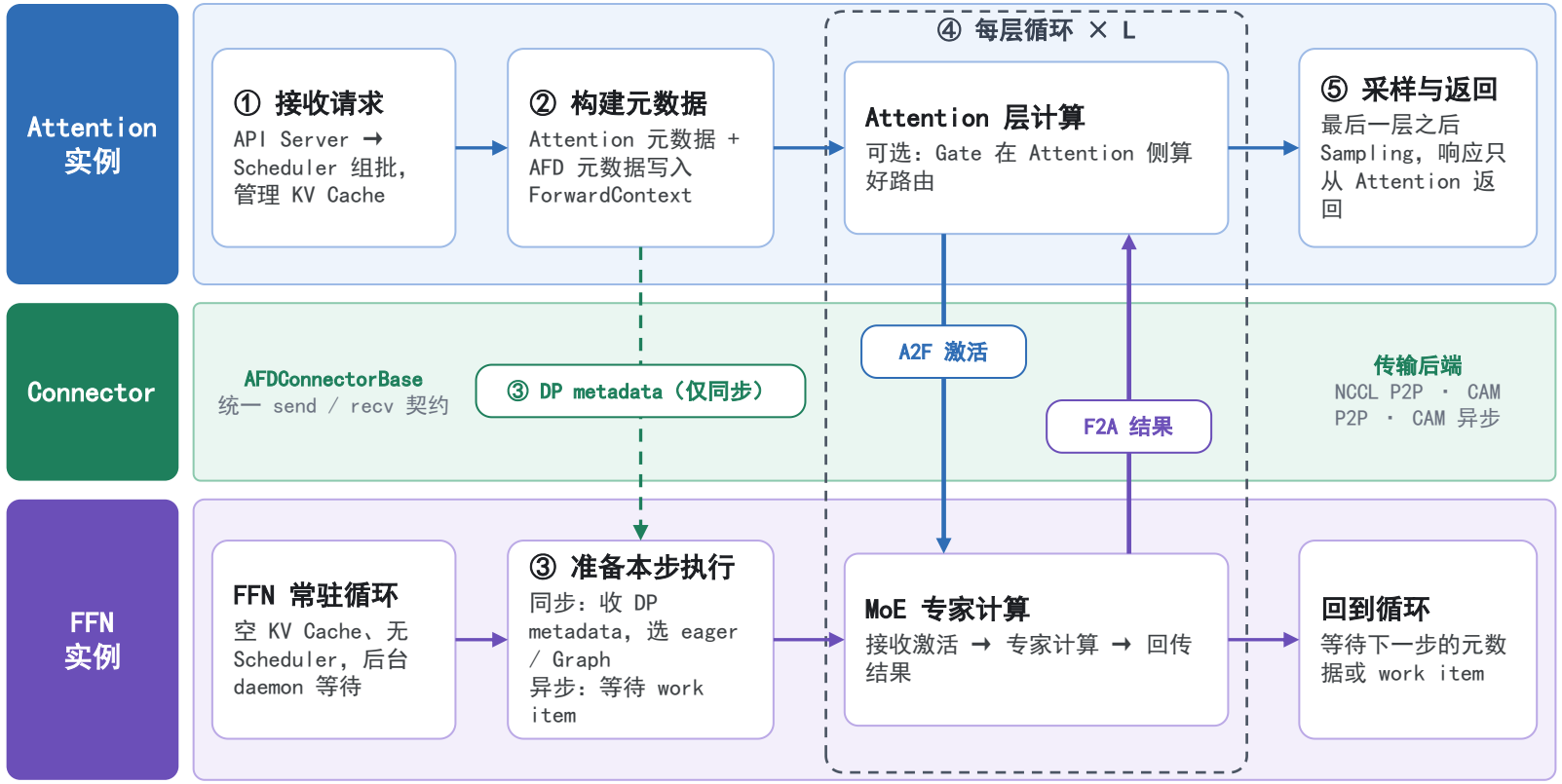
vLLM 0.30.0 / vLLM-Ascend
复用上游运行时，不修改源码

OpenAI API Server | EngineCore · Scheduler | ModelRunner V2 / V1 | Layers · Ops | KV Cache

插件只新增角色运行时、Connector 与模型封装；与上游的差异收敛在 compat/ 与原生算子中，vLLM 源码保持不变。

来源：afd_plugin/__init__.py、v1/worker、connectors、model_executor、compat；README.md（vLLM 0.30.0）

运行时数据流：Attention 驱动请求，FFN 逐层应答



请求只发给 Attention 的 API Server · 两侧可按任意顺序启动 · FFN 拒绝 scheduler 驱动的 execute_model

同步 Connector

P2pNccl（CUDA） · CAMP2p（Ascend NPU）

- 每一步先经控制面下发 DP metadata，以及 warmup / capture / replay 标记
- FFN 按 stage 形状执行；Graph：CUDA 为 FULL_DECODE_ONLY，Ascend V2 另支持 FULL
- 支持 DBO：两个微批交替通信与计算

异步 Connector

CAMAsync（Ascend NPU）

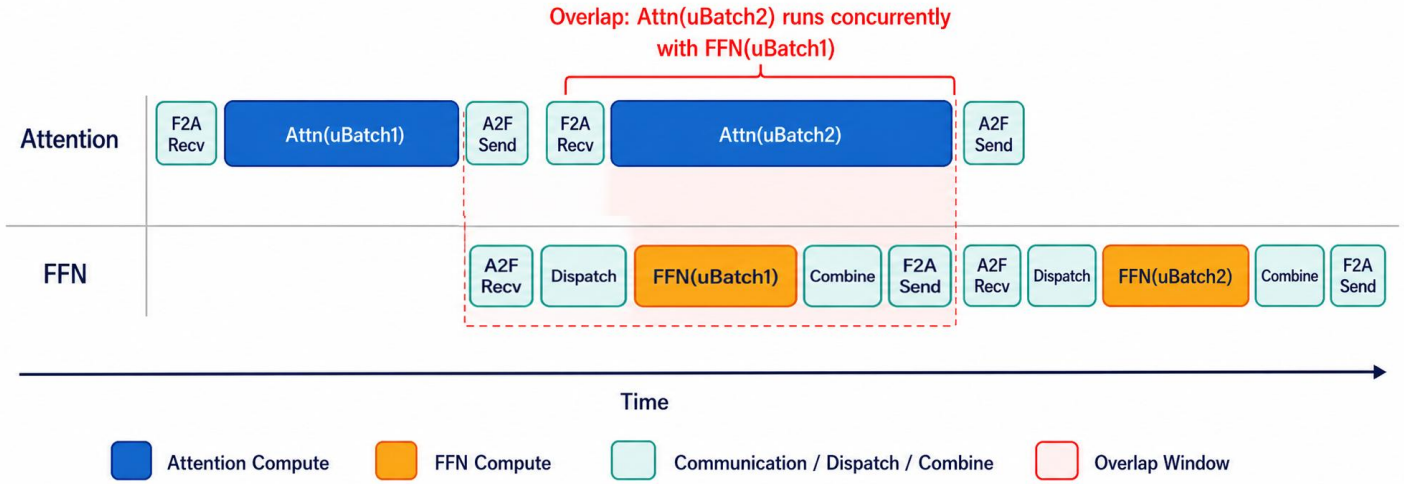
- 无控制面：Gate 在 Attention 侧，top-k 路由随 dispatch 一起发出
- FFN 阻塞等待 work item，按专家区间计算，经 combine 回传
- 仅 eager；DBO 只在 Attention 侧开启（含 DP1）

同一套模型代码、两种驱动方式：同步路径靠控制面对齐 batch 形状，异步路径靠 dispatch payload 携带路由。

来源：docs/design/module/attention_runtime.md · ffn_runtime.md · connector_contracts.md; README.md

CAMP2PConnector：双微批完全重叠

缩短 FFN 计算段后，下一批 Attention 与 FFN 同步启动，并在 FFN 结束后继续执行，完整隐藏当前批 FFN 时延。



流水过程

双微批持续推进，通信与两类计算交叠执行

1. **F2A 接收与 Attention**
   Attention 侧接收上一层 FFN 返回的激活，执行 uBatch1。
2. **A2F 发送**
   uBatch1 完成后立即 P2P 发往 FFN，同时启动 uBatch2。
3. **FFN 完整被覆盖**
   FFN(uBatch1) 完整落在 Attention(uBatch2) 时间窗内。
4. **F2A 返回并进入下一轮**
   FFN 输出发回 Attention；两侧继续处理后续微批。

关键重叠　　**Attention(uBatch2) 完整覆盖 FFN(uBatch1)**

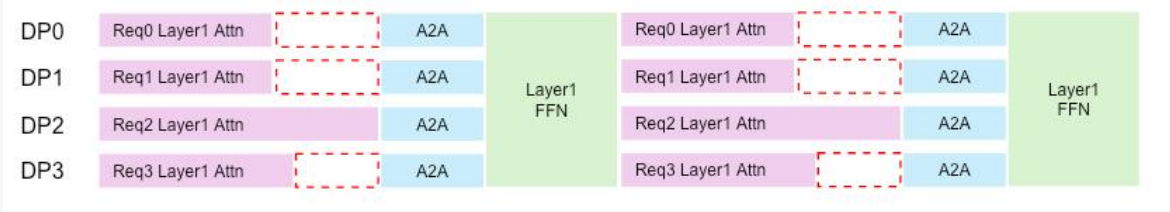
Attention 与 FFN 同步启动并延后结束；通信、专家计算与返回阶段持续流水化，减少同步等待。

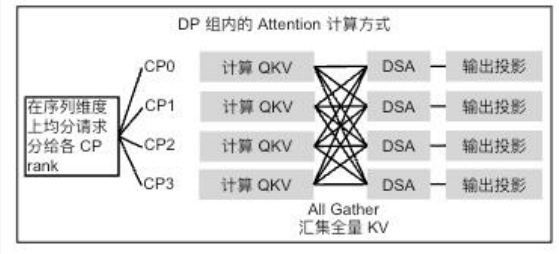
DBO 将通信嵌入双微批流水，Attention 的执行窗口完整覆盖当前批 FFN

来源：afd-plugin/docs/npu/dbo.png；流水图由 ImageGen 在原图语义上定向润色

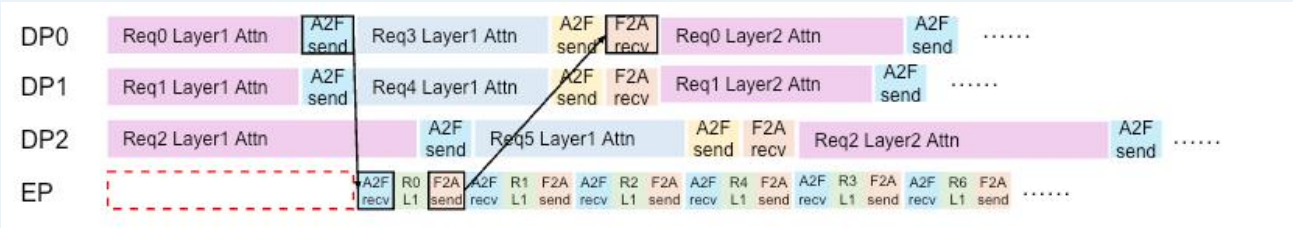
Attention－FFN Prefill 分离异步方案

01 同步方案：AF不分离，DP调度不平衡，需等待最慢卡





02 AFD异步方案：AF分离，无需等待最慢卡，Attention算完即可继续执行



Attention 与 FFN 独立推进

A2F send / F2A recv 穿插在请求流水中，EP 侧连续处理各请求的专家计算。

03 异步通信算子

dispatch_send Attention卡根据路由表将每个token按top-k选择分发到对应MoE Expert卡的共享内存中，同时均匀分配token给各卡的共享专家。

combine_send MoEExpert卡完成本地专家计算后，根据dispatch_recv产出的元信息将路由专家和共享专家的计算结果按原路写回对应Attention卡的共享内存。

dispatch_recv MoEExpert卡轮询各Attention卡的共享内存，等待数据就绪后将token按专家维度重排收集到本地buffer，输出每个专家收到的token及元信息供后续计算使用。

combine_recv Attention卡等待所有MoE Expert卡回传完成，遍历路由表从共享内存中读出每个token的top-k专家输出与共享专家输出，按门控分数加权累加得到最终token输出。

同步与异步方案对比；四个通信算子说明保持原材料内容

来源：AFD-Plugin(2).pptx 第 6 页

03

部署方式与性能结果



Usage：分别启动 Attention 与 FFN 实例

01 GPU Attention-side shape

```
vllm serve /path/to/DeepSeek-V2-Lite \
  --served-model-name deepseek-v2-lite-afd-attention \
  --data-parallel-size 1 \
  --tensor-parallel-size 1 \
  --enable-expert-parallel \
  --enforce-eager \
  --host 127.0.0.1 \
  --port 18000 \
  --additional-config
'{"afd":{"role":"attention","connector":"P2pNcclAFDConnector","host":"127.0.0.1","port":6239,"num_attention_ranks":1,"num_ffn_ranks":1}}'
```

02 GPU FFN-side shape

```
vllm serve /path/to/DeepSeek-V2-Lite \
  --served-model-name deepseek-v2-lite-afd-ffn \
  --data-parallel-size 1 \
  --tensor-parallel-size 1 \
  --enable-expert-parallel \
  --enforce-eager \
  --host 127.0.0.1 \
  --port 18001 \
  --additional-config '{"afd":{"role":"ffn","connector":"P2pNcclAFDConnector","host":"127.0.0.1","port":6239,"num_attention_ranks":1,"num_ffn_ranks":1}}'
```

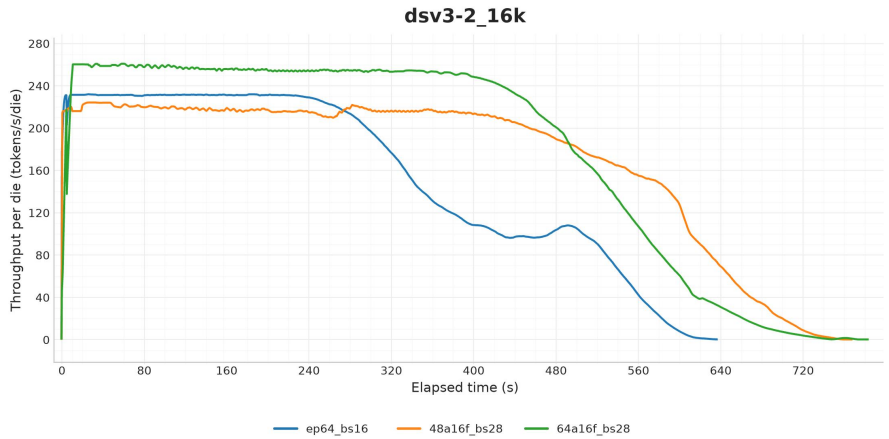
同一组 host / port 与 rank 数配置必须在两侧保持一致

来源：AFD Plugin README · GPU P2P serving example

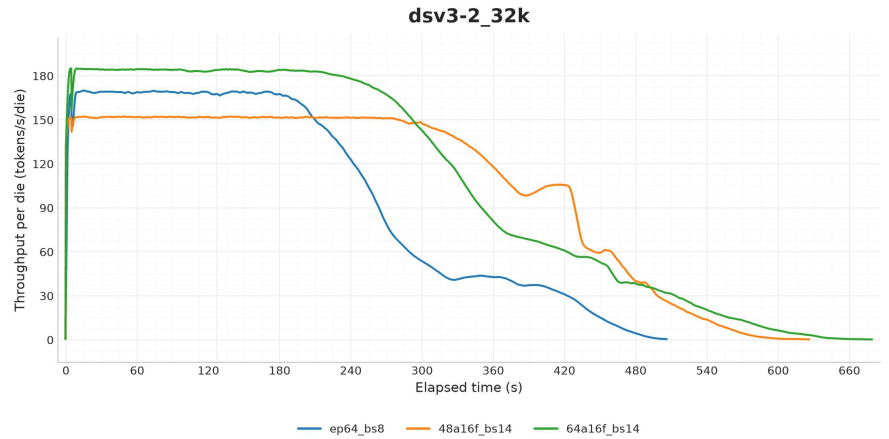
AFD同步Decode场景：CAMP2PConnector

测试口径：版本 v0.19.1rc1；DeepSeek-V3.2 W8A8、Ascend 910C，输出长度 512－1,536 tokens，吞吐按物理 die 归一化。结果使用 forced expert balancing，仅用于受控性能对比，不代表精度或生产 serving。

16K input



32K input



Input	EP64	48A16F	Δ vs EP64	64A16F	Δ vs EP64
16K	232.6	220.3	−5.3%	258.9	+11.3%
32K	168.2	151.4	−10.0%	183.3	+9.0%

解耦提供了资源配比的调参空间，但不保证任意 A:F 比例都更快

测试设置：版本 v0.19.1rc1；开启专家强制均衡（forced expert balancing）单位：output tokens/s/die

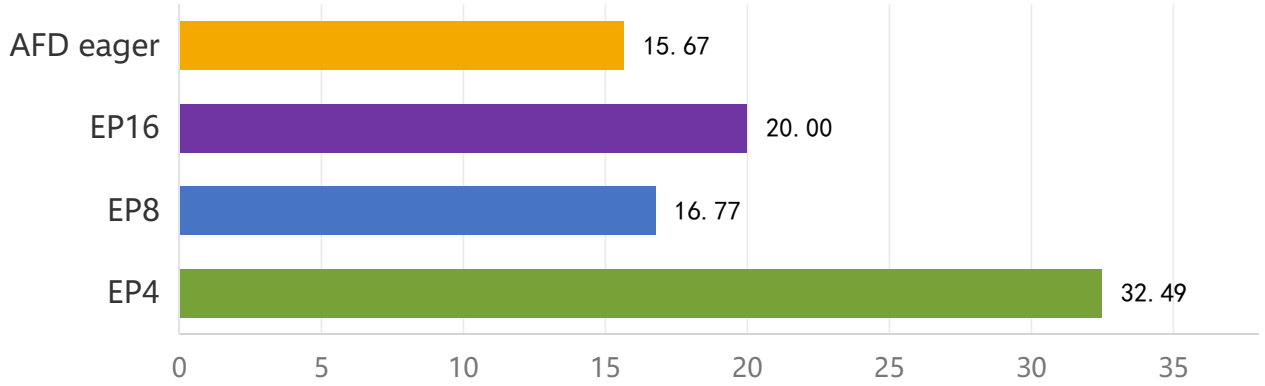
Prefill异步AFD CAMAsyncConnector Agentic 多轮会话场景

模型：DeepSeek V4 Flash W4A8

硬件：Ascend 910C · 同机 16 DIE

128 个完整会话 · 3,725 个请求 | 最多 12 个在途请求 | 前缀缓存开启

完整负载耗时（分钟）



首 token 延迟 TTFT（秒）

配置	均值	P50	P95	P99
EP4	6.239	2.722	24.768	39.354
EP8	3.171	2.831	6.532	9.552
EP16	3.793	3.486	6.676	8.743
AFD	**2.936**	**2.531**	**6.361**	**8.611**

比 EP4 少用时 51.8%
32.49 → 15.67 分钟

比 EP8 少用时 6.5%
16.77 → 15.67 分钟

比 EP16 少用时 21.7%
20.00 → 15.67 分钟

部署	实例 / 角色	并行配置
EP4	4 个实例，每实例 4 DIE	每实例 DP1 / TP4 / EP4
EP8	2 个实例，每实例 8 DIE	每实例 DP2 / TP4 / EP8
EP16	1 个实例，共 16 DIE	DP4 / TP4 / EP16
AFD	**12A + 4F**	**A：DP3 / TP4；F：DP4 / TP1 / EP4**

测试设置：Agentic 会话历史回放，每轮输出 1 token；固定会话路由、异步调度；各组单次冷缓存运行。
单步 token 上限：Non 8,192 / AFD 32,768；AFD 使用 CAM 异步通信与两份 token 微批次。2026-10-14 实测。

04

社区进展与后续规划



未来规划：六个方向推动 AFD 持续演进

01 更多模型支持

现状 DeepSeek V2/V3/V3.2/V4、GLM-MoE-DSA、Qwen3 / Qwen3.5 / 3.6 MoE

- 扩展更多 MoE 模型族（如 Kimi K3），补齐 Qwen 系列的 NPU 支持
- 用 adapt-model Skill 固化适配与验证，每个模型配 Recipe 与 E2E 精度门禁

02 FFN 侧轻量服务化，独立演进

现状 FFN 仍经 vllm serve 启动，靠 EngineCore Patch 跳过 Scheduler / KV Cache

- 独立 FFN Service：只加载 FFN 权重，统一 step loop、warmup 与 Graph（#391）
- 版本化 A/F 协议，探索 Attention 与 FFN 独立升级、独立运维

03 跟随上游 vLLM 版本

现状 精确绑定 vLLM 0.30.0（Ascend 8d4409d6），已完成 0.26 → 0.30 升级

- 按上游版本节奏跟随；推动上游开放 AFD 所需扩展点
- 升级 Skill 固化版本审计、Patch 处置与 E2E 回归，持续收敛 Patch

04 异构硬件，提升性价比

现状 已支持 CUDA 与 Ascend NPU，但 A/F 两侧仍为同构部署

- 按负载特征为 Attention / FFN 选用性价比最优的硬件，沉淀降本实践
- 探索 FFN 专用硬件（如 LPU）与跨架构 Connector

05 Decode 阶段异步化探索

现状 Decode 以同步 Connector 为主（Graph + DBO）；异步 CAM 已能跑 Decode，但仅 eager

- Decode 引入异步调度，消除逐层全局同步屏障
- 兼顾 Graph 与 ITL 稳定性，与“同步 + DBO”方案对比收益

06 非标准 AF 切分方式

现状 在 MoE / MLP 边界切分；Gate、共享专家、Dense 层的位置随模型与模式而定

- 更灵活的切分点：部分层共置，共享专家、Dense 层的放置可配置
- 结合模型结构与硬件，按层差异化切分与 A:F 配比

目标：让 AFD 从“可运行的外部插件”演进为可复现、可扩展、与上游协同的 MoE Serving 能力

方向 02 已有公开讨论：issue #391（FFN as a Service）；其余方向欢迎在 GitHub / Slack 参与共建

来源：vllm-project/afd-plugin README.md、issue #391、docs/design/module/execution_platforms.md（2026-10-15）

AFD Plugin Community

01 项目首页

Code · Docs · Issues

github.com/vllm-project/afd-plugin

查看代码、架构文档与最新进展

02 Developer Slack

Discussion · Collaboration

vllm-dev.slack.com/archives/C0B4C1D84GG

加入 vLLM Developer Slack 后访问频道

03 WeChat 社群

Chinese Developer Community

afd-plugin 开发者交流群

二维码取自项目仓库，请以最新版本为准

GOSIM

GOSIM Shenzhen 2026

Thanks!

Q&A

github.com/vllm-project/afd-plugin



GOSIM

GOSIM Shenzhen 2026

Thanks!

Q&A

github.com/vllm-project/afd-plugin



GOSIM

GOSIM Shenzhen 2026

Thanks!

Q&A

github.com/vllm-project/afd-plugin

